

```

import os
import sys
import numpy as np
import librosa
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Dropout,Activation,Flatten
from tensorflow.keras.optimizers import Adam
from sklearn import metrics
import tensorflow.keras
import tensorflow.keras.layers as layers
import IPython.display as ipd
from tensorflow.keras.callbacks import ReduceLROnPlateau
import matplotlib.pyplot as plt

```

```

print("Python version:", sys.version)
print("NumPy version:", np.__version__)
print("librosa version:", librosa.__version__)
print("TensorFlow version:", tf.__version__)

```

```

Python version: 3.8.20 (default, Oct 3 2024, 15:19:54) [MSC v.1929 64 bit (AMD64)]
NumPy version: 1.19.5
librosa version: 0.9.2
TensorFlow version: 2.5.3

```

```

def preprocess_file(song_path, n_mfcc=128, num_segments=14):
    """
    Preprocess a single song into MFCC feature segments.
    Each segment is reshaped to (16, 8, 1).
    """
    mfcc_segments = []

    try:
        duration = librosa.get_duration(filename=song_path)

        if duration > 90:
            audio, sr = librosa.load(
                song_path,
                sr=None,
                offset=30,
                duration=60
            )
        else:
            audio, sr = librosa.load(song_path, sr=None)

        total_samples = len(audio)
        segment_length = total_samples // num_segments

        for s in range(num_segments):
            start = s * segment_length
            end = start + segment_length
            segment = audio[start:end]

            if len(segment) < segment_length // 2:
                continue

            mfcc = librosa.feature.mfcc(y=segment, sr=sr, n_mfcc=n_mfcc)
            mfcc_mean = np.mean(mfcc.T, axis=0)
            mfcc_resaped = np.reshape(mfcc_mean, (16, 8, 1))
            mfcc_segments.append(mfcc_resaped)

    except Exception as e:
        print(f"Error processing {song_path}: {e}")

    return mfcc_segments

def preprocess_dataset(parent_dir, genres, n_mfcc=128, num_segments=14, max_files=None):
    dataset = []
    labels = []

    for genre, genre_number in genres.items():
        genre_dir = os.path.join(parent_dir, genre)
        print(f"Processing genre: {genre}")

```

```

files = [f for f in os.listdir(genre_dir) if f.endswith((".wav", ".au", ".mp3"))]
if max_files:
    files = files[:max_files]

for filename in files:
    song_path = os.path.join(genre_dir, filename)
    mfcc_segments = preprocess_file(song_path, n_mfcc, num_segments)

    dataset.extend(mfcc_segments)
    labels.extend([genre_number] * len(mfcc_segments))

dataset = np.array(dataset)
labels = np.array(labels)

print("Final dataset shape:", dataset.shape)
print("Final labels shape:", labels.shape)
return dataset, labels

```

```

Xcheck= preprocess_file("data/genresWAV/classical/classical.00003.wav")
print(np.array(Xcheck).shape)

```

```
(14, 16, 8, 1)
```

```

genres = {
    'blues': 0, 'classical': 1, 'country': 2, 'disco': 3, 'hiphop': 4,
    'jazz': 5, 'metal': 6, 'pop': 7, 'reggae': 8, 'rock': 9
}

```

```
X, y = preprocess_dataset("data/genresWAV", genres)
```

```

Processing genre: blues
Processing genre: classical
Processing genre: country
Processing genre: disco
Processing genre: hiphop
Processing genre: jazz
Processing genre: metal
Processing genre: pop
Processing genre: reggae
Processing genre: rock
Final dataset shape: (14000, 16, 8, 1)
Final labels shape: (14000,)

```

```
X.shape
```

```
(14000, 16, 8, 1)
```

```

labelencoder = LabelEncoder()
y = to_categorical(labelencoder.fit_transform(y))

from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=0)

```

```

INPUTSHAPE = (16,8,1)
model = Sequential([

    layers.Conv2D(32 , (3,3),activation = 'relu',padding='valid', input_shape = INPUTSHAPE),
    layers.MaxPooling2D(2, padding='same'),
    layers.Flatten(),
    layers.Dense(512, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(1e-4)),
    layers.Dense(10 , activation = 'softmax')
])

model.compile(loss = 'categorical_crossentropy', optimizer
= 'adam'
, metrics =['accuracy'])
model.summary()

```

```
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 14, 6, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 7, 3, 32)	0
flatten_2 (Flatten)	(None, 672)	0
dense_4 (Dense)	(None, 512)	344576

```
dense_5 (Dense)          (None, 10)          5130
=====
Total params: 350,026
Trainable params: 350,026
Non-trainable params: 0
```

```
batch_size = 128
callback = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', min_delta=0, patience=8, verbose=0, mode='auto',
    baseline=None, restore_best_weights=False)

lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=1)
#checkpointer = ModelCheckpoint(filepath='saved_models/audio_classification.hdf5',
    # verbose=1, save_best_only=True)

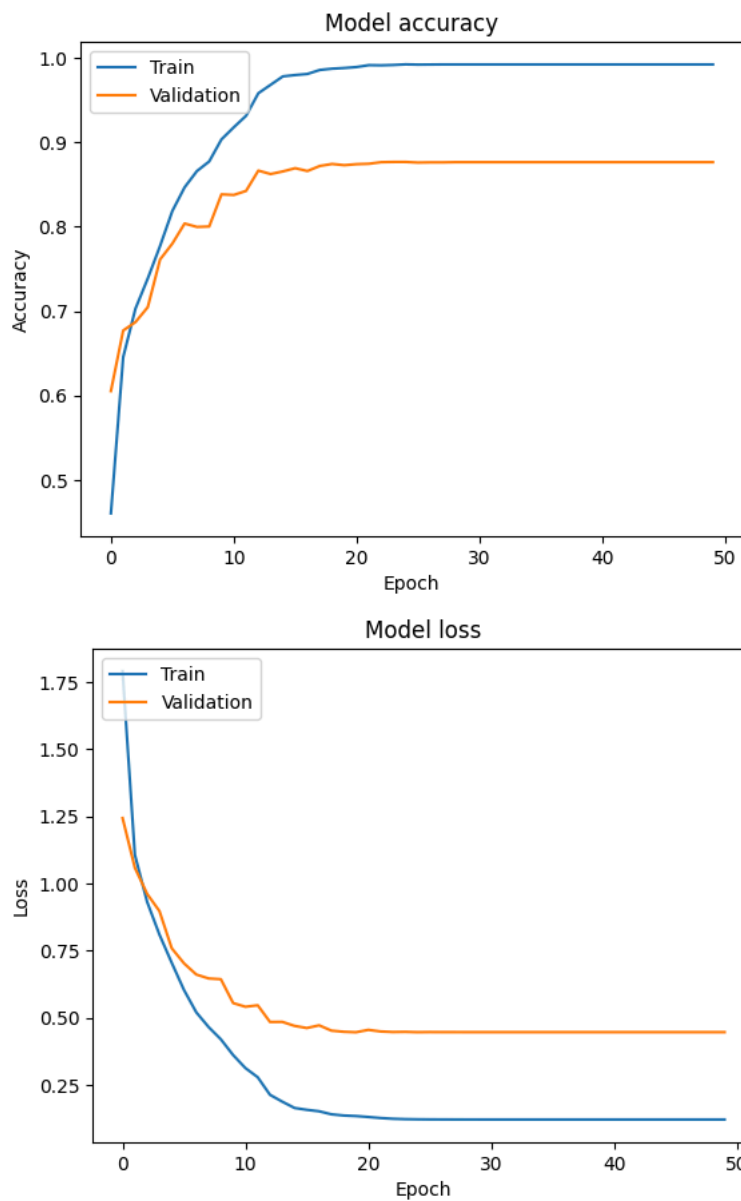
history = model.fit(X_train,y_train ,validation_data=(X_test,y_test),
    epochs=50,
    verbose=1,
    callbacks = [lr_scheduler],batch_size=batch_size)
```

```
Epoch 1/50
77/77 [=====] - 2s 13ms/step - loss: 1.9405 - accuracy: 0.4457 - val_loss: 1.2874 - val_accuracy:
Epoch 2/50
77/77 [=====] - 1s 11ms/step - loss: 1.1382 - accuracy: 0.6240 - val_loss: 1.1594 - val_accuracy:
Epoch 3/50
77/77 [=====] - 1s 10ms/step - loss: 0.9431 - accuracy: 0.6867 - val_loss: 0.9499 - val_accuracy:
Epoch 4/50
77/77 [=====] - 1s 10ms/step - loss: 0.8028 - accuracy: 0.7486 - val_loss: 0.8415 - val_accuracy:
Epoch 5/50
77/77 [=====] - 1s 11ms/step - loss: 0.7312 - accuracy: 0.7670 - val_loss: 0.7883 - val_accuracy:
Epoch 6/50
77/77 [=====] - 1s 12ms/step - loss: 0.6047 - accuracy: 0.8191 - val_loss: 0.6934 - val_accuracy:
Epoch 7/50
77/77 [=====] - 1s 13ms/step - loss: 0.5263 - accuracy: 0.8427 - val_loss: 0.7526 - val_accuracy:
Epoch 8/50
77/77 [=====] - 1s 13ms/step - loss: 0.4285 - accuracy: 0.8810 - val_loss: 0.5912 - val_accuracy:
Epoch 9/50
77/77 [=====] - 1s 12ms/step - loss: 0.3895 - accuracy: 0.8957 - val_loss: 0.5938 - val_accuracy:
Epoch 10/50
77/77 [=====] - 1s 12ms/step - loss: 0.3376 - accuracy: 0.9195 - val_loss: 0.5439 - val_accuracy:
Epoch 11/50
77/77 [=====] - 1s 12ms/step - loss: 0.3186 - accuracy: 0.9286 - val_loss: 0.5443 - val_accuracy:
Epoch 12/50
77/77 [=====] - 1s 12ms/step - loss: 0.2945 - accuracy: 0.9373 - val_loss: 0.5303 - val_accuracy:
Epoch 13/50
77/77 [=====] - 1s 11ms/step - loss: 0.2870 - accuracy: 0.9420 - val_loss: 0.5236 - val_accuracy:
Epoch 14/50
77/77 [=====] - 1s 11ms/step - loss: 0.2809 - accuracy: 0.9440 - val_loss: 0.5212 - val_accuracy:
Epoch 15/50
77/77 [=====] - 1s 13ms/step - loss: 0.2749 - accuracy: 0.9476 - val_loss: 0.5274 - val_accuracy:
Epoch 16/50
77/77 [=====] - 1s 14ms/step - loss: 0.2631 - accuracy: 0.9503 - val_loss: 0.5108 - val_accuracy:
Epoch 17/50
77/77 [=====] - 1s 14ms/step - loss: 0.2589 - accuracy: 0.9529 - val_loss: 0.5083 - val_accuracy:
Epoch 18/50
77/77 [=====] - 1s 13ms/step - loss: 0.2549 - accuracy: 0.9537 - val_loss: 0.5051 - val_accuracy:
Epoch 19/50
77/77 [=====] - 1s 13ms/step - loss: 0.2515 - accuracy: 0.9557 - val_loss: 0.5109 - val_accuracy:
Epoch 20/50
77/77 [=====] - 1s 13ms/step - loss: 0.2461 - accuracy: 0.9572 - val_loss: 0.5031 - val_accuracy:
Epoch 21/50
77/77 [=====] - 1s 14ms/step - loss: 0.2436 - accuracy: 0.9589 - val_loss: 0.5000 - val_accuracy:
Epoch 22/50
77/77 [=====] - 1s 15ms/step - loss: 0.2415 - accuracy: 0.9607 - val_loss: 0.5019 - val_accuracy:
Epoch 23/50
77/77 [=====] - 1s 13ms/step - loss: 0.2386 - accuracy: 0.9606 - val_loss: 0.5011 - val_accuracy:
Epoch 24/50
77/77 [=====] - 1s 12ms/step - loss: 0.2370 - accuracy: 0.9621 - val_loss: 0.4985 - val_accuracy:
Epoch 25/50
77/77 [=====] - 1s 14ms/step - loss: 0.2365 - accuracy: 0.9618 - val_loss: 0.4989 - val_accuracy:
Epoch 26/50
77/77 [=====] - 1s 15ms/step - loss: 0.2357 - accuracy: 0.9616 - val_loss: 0.4985 - val_accuracy:
Epoch 27/50
77/77 [=====] - 1s 13ms/step - loss: 0.2351 - accuracy: 0.9618 - val_loss: 0.4985 - val_accuracy:
Epoch 28/50
77/77 [=====] - 1s 12ms/step - loss: 0.2349 - accuracy: 0.9617 - val_loss: 0.4984 - val_accuracy:
Epoch 29/50
77/77 [=====] - 1s 12ms/step - loss: 0.2349 - accuracy: 0.9617 - val_loss: 0.4984 - val_accuracy:
```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
```

```
plt.show()

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```



```
test_accuracy=model.evaluate(X_test,y_test,verbose=0)
print(test_accuracy[1])
```

```
0.8604761958122253
```

```
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix

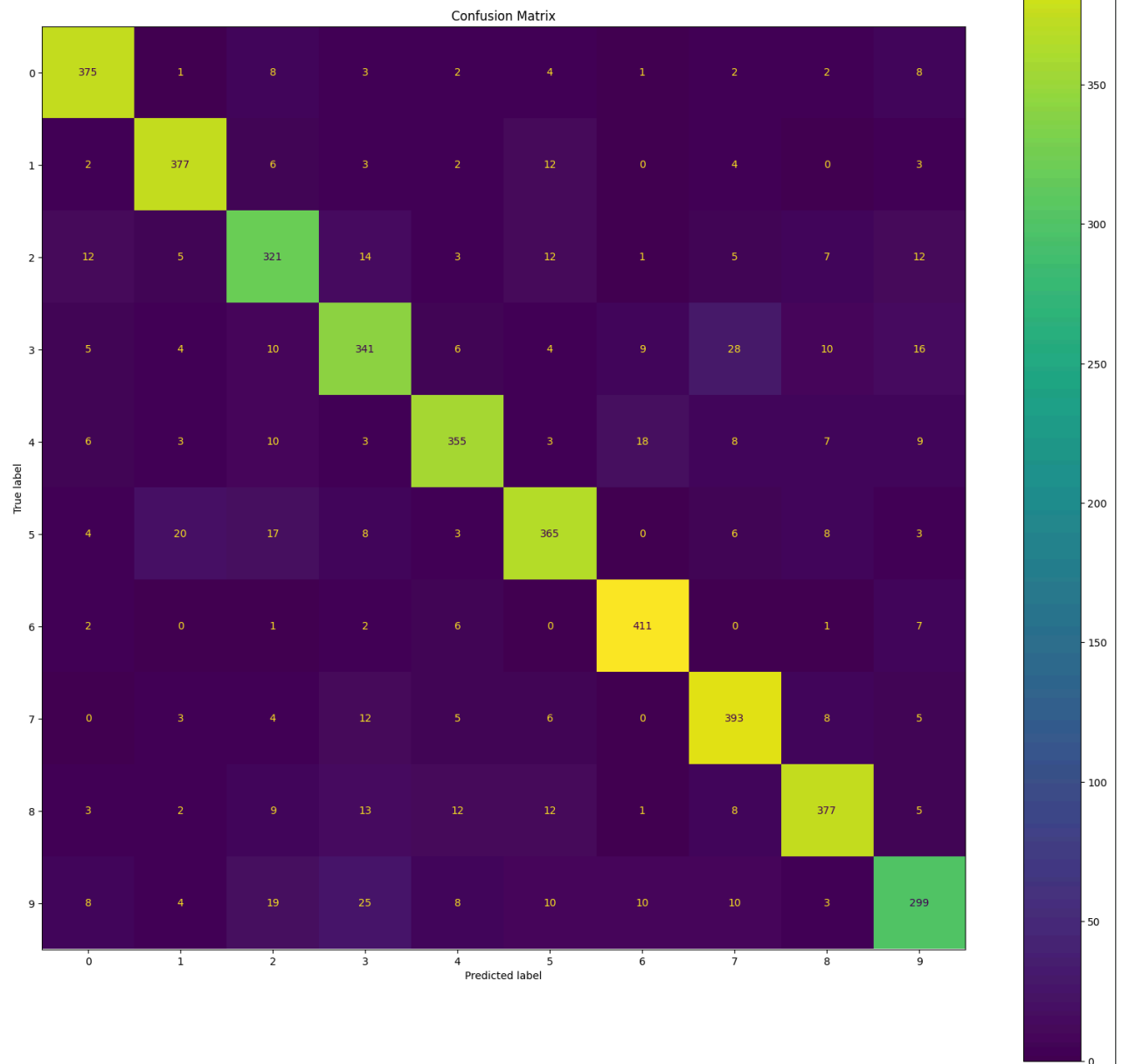
y_pred_prob = model.predict(X_test, verbose=0)

y_true = np.argmax(y_test, axis=1)
y_pred = np.argmax(y_pred_prob, axis=1)
cm = confusion_matrix(y_true, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

fig, ax = plt.subplots(figsize=(20, 20))
disp.plot(ax=ax)

plt.title("Confusion Matrix")
plt.show()
```



```
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    classification_report
```

```

)

accuracy = accuracy_score(y_true, y_pred)

precision = precision_score(y_true, y_pred, average='weighted')
recall = recall_score(y_true, y_pred, average='weighted')
f1 = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")

```

```

Accuracy : 0.8605
Precision: 0.8601
Recall : 0.8605
F1 Score : 0.8599

```

```

import seaborn as sns
from sklearn.metrics import ConfusionMatrixDisplay

y_pred=best_model.predict(X_test)
round_off=np.round_(y_pred)
cm=confusion_matrix(y_test.argmax(axis=1),round_off.argmax(axis=1))
print("Confusion Matrix")
print(cm)

plt.figure(figsize=(20,20))
sns.heatmap(cm,annot=True,fmt="d",cmap='Set3')
plt.title("Confusion Matrix")
plt.ylabel("True class")
plt.xlabel("Predicted class")
plt.show()

```

```

##test this
mydict = {
    'Alternative': 0, 'AlternativeHeavy': 1, 'Electronica': 2, 'Metal': 3, 'Punk': 4,
    'Rap': 5, 'Rave': 6, 'Reggae': 7, 'Rock': 8, 'Stoner': 9
}
xtest = np.array(preprocess_file('test/The Mamas & The Papas - California Dreamin.mp3'))
y_pre =model.predict(xtest)
pred_class = np.argmax(y_pre, axis=1)[0]
#print(pred_class)
for genre, classs in mydict.items():
    if classs == pred_class:
        print("Predicted Genre:", genre)

```

```

model = tf.keras.models.load_model("THESIS_MFCC.h5")
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

with open("THESIS_MFCCTHESIS_MFCC", "wb") as f:
    f.write(tflite_model)

print("Saved Model successfully.")

```

Below Code is based On works of

<https://github.com/Manishankar9977/Music-genre-classification/blob/main/music%20gerne%20with%20cnn.ipynb>

another PSO trained model is:

```

INPUTSHAPE = (16,8,1)
model = Sequential([

    layers.Conv2D(55 , (3,3),activation = 'relu',padding='valid', input_shape = INPUTSHAPE),
    layers.MaxPooling2D(2, padding='same'),
    layers.Flatten(),
    layers.Dense(313, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(1e-4)),
    layers.Dense(10 , activation = 'softmax')

])

model.compile(loss = 'categorical_crossentropy', optimizer
=tf.keras.optimizers.Adam(learning_rate=0.0007842025145903583)
# = 'adam'

```

```
, metrics=['accuracy'])  
model.summary()
```